Student Management System

Add Student & View Student Modules — Concept and Flow

This report explains, in simple terms, how the Add Student and View Student features work in a PHP and MySQL based system. It covers the underlying web concepts (CRUD, HTTP methods, client-server architecture) along with the process flow diagram and the actual PHP code.

1. What is CRUD?

Almost every data-driven application performs four basic operations on data, commonly known as **CRUD**:

Operation	Meaning	Example in this system
Create	Add new data	Add Student form
Read	View / retrieve data	View Students page
Update	Modify existing data	Edit Student (future module)
Delete	Remove data	Delete Student (future module)

The current system implements the **Create** and **Read** operations through two PHP files: `add_student.php` and `view_students.php`.

2. HTTP Request Methods

Every time a browser communicates with a server, it uses an HTTP method. The two most common methods used in this system are:

Method	Purpose	Used In
GET	Requesting / viewing data	Opening view_students.php
POST	Sending data to the server	Submitting the Add Student form

This is why the code checks `$_SERVER['REQUEST_METHOD'] === 'POST'` before inserting data — it ensures the insert logic runs only when the form has actually been submitted, not simply when the page loads.

3. Client-Server-Database Architecture

A typical PHP + MySQL web application follows a three-layer architecture:

- **Client (Browser):** Displays the HTML form, collects user input.
- **Server (PHP):** Acts as the middle layer — receives input, validates it, and talks to the database.
- **Database (MySQL):** Stores the data permanently in tables.

PHP itself does not store any data — it only sends instructions (SQL queries) to MySQL, which is responsible for permanent storage.

4. Prepared Statements (Security)

Instead of directly inserting user input into an SQL query, prepared statements use placeholders (?) and bind values separately:

```
$stmt = $conn->prepare("INSERT INTO students (name) VALUES (?)");
$stmt->bind_param("s", $name);
```

This approach prevents **SQL Injection**, a common attack where malicious code is inserted through form fields to manipulate or damage the database.

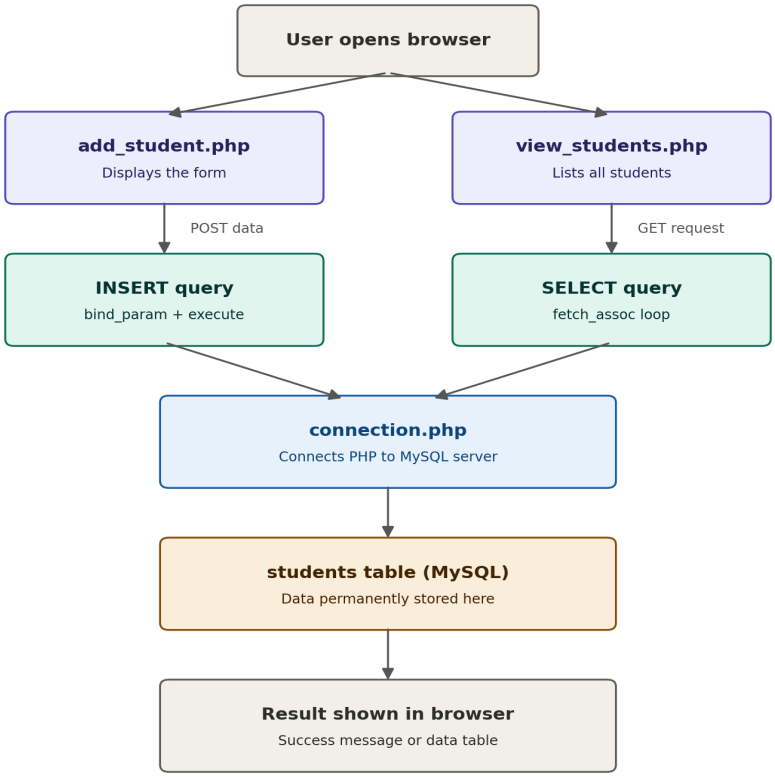
5. Statelessness of HTTP

HTTP is a **stateless** protocol — each request is independent, and the server does not remember the previous request. This is why:

- After adding a student, the code uses `header("Location: view_students.php")` to redirect the browser.
- The actual data is stored permanently in the MySQL database (not in memory), so it remains visible on the View page even after the request ends.

6. Process Flow Diagram

The diagram below shows how the Add Student and View Student modules both connect to the same database through a shared connection file.



Summary of the Flow

Step	Add Student	View Student
HTTP Method	POST	GET
SQL Query	INSERT	SELECT
Data Direction	Form to Database	Database to Browser
Purpose	Create new record	Read / display records

7. PHP Code Reference

connection.php

```
<?php
$conn = new mysqli("localhost", "root", "", "school_db");
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
?>
```

add_student.php

```
<?php
require "connection.php";

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name = $_POST['name'];
    $roll_number = $_POST['roll_number'];
    $class = $_POST['class'];
    $phone = $_POST['phone'];

    $stmt = $conn->prepare("INSERT INTO students (name, roll_number, class, phone) VALUES
(?, ?, ?, ?)");
    $stmt->bind_param("ssss", $name, $roll_number, $class, $phone);
    $stmt->execute();
}
?>
```

view_students.php

```
<?php
require "connection.php";
$result = $conn->query("SELECT * FROM students ORDER BY id DESC");
?>

<?php while ($row = $result->fetch_assoc()): ?>
    <tr><td><?= $row['name'] ?></td></tr>
<?php endwhile; ?>
```